2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA) | 979-8-3503-4235-2/23/$31.00 ©2023 IEEE | DOI: 10.1109/SEAA60479.2023.00047

An Exploratory Case Study: Using Petri Nets for Modelling Microservice-based Systems

Görkem Kılınç Soylu
Department of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey
gorkemkilinc@iyte.edu.tr

Onur Demirörs
Department of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey
onurdemirors@iyte.edu.tr

***Abstract*—The rapid evolution of cloud technology has led to the development of highly responsive and flexible applications. Microservice-based Architecture (MSbA) is a popular approach for creating cloud-compatible applications. However, MSbA presents challenges in system verification and analysis. This paper explores using Petri nets for modelling microservice-based systems. The proposed method utilises abstraction and composition techniques to model complex systems using 1-safe Petri nets. A case study on Banking as a Service (BaaS) demonstrates the suitability of Petri nets for modelling, analysis and verification of microservice-based systems.**

***Index Terms*—Microservice, modelling, analysis, verification, Petri nets.**

I. Introduction

The rapid advancements in cloud technology, along with the increasing demand for continuous digital transformation, have led to the emergence of modern applications that are highly responsive, flexible, and reliable. In order to meet these evolving requirements, new software design paradigms and architectures have emerged, with a specific focus on developing cloud-compatible applications. One such approach gaining popularity among leading software design companies is Microservice-based Architecture (MSbA) [1], [2]. MSbA is an architectural style that emphasises the decomposition of an application into a collection of loosely coupled and independently deployable services known as microservices. Each microservice represents a specific business process and can be developed, deployed, and scaled independently. This modular approach allows for greater agility, scalability, and fault tolerance in the application. The state of the art on MSbA is provided in [3], [4].

While providing benefits for software engineering, MSbA approach also presents several challenges due to the complex and distributed nature. The authors address such challenges based on a survey performed with organisations working with microservices in [5]. Some of these challenges specifically effect software management such as modelling, testing and verification. New approaches are needed to overcome the challenges introduced by this paradigm shift in software engineering and management.

Formal methods can be used to tackle these challenges. The benefits of applying formal methods for verification of microservice-based systems are that they provide a rigorous and systematic approach to modelling and analysing complex distributed applications. Formal methods can help identify potential errors or vulnerabilities in the system design before implementation, which can save time and resources in the long run. Formal methods are able to guarantee system correctness given that the application complies with the model. Formal methods can also help for generating test cases by computing reachable states and identifying critical ones.

Providing tailored modelling techniques for MSbA is essential to overcome the above mentioned challenges. The authors of [5], [6] and [3] provide insights for the modelling techniques used in the organisations working with microservices and presents their shortcomings. The most used techniques include using UML diagrams such as use case diagrams, class diagrams, activity diagrams. Other techniques used in modelling microservices are using eEPC diagrams [6] and BPMN [7]. However all these models lack formal semantics which can result in incorrect interpretations of the models. Also, these models cannot be formally verified. Having formal semantics of models provides clarity and allows formal analysis and verification. Although they lack formal semantics, eEPC [8] and BPMN [9] are in reality related to Petri nets. Thus, the models created with eEPC and BPMN can be translated to Petri nets and thus existing formal methods and tools provided for Petri nets can be used for verification.

In this work, we propose using Petri nets for modelling microservice-based systems. Petri nets are known to be powerful for modelling and analysing distributed systems. There is a well studied and strong theory behind and a rich collection of tools exist for supporting modelling and analysis. Petri nets are particularly promising in modelling microservice-based systems due to their power of representing both synchronous and asynchronous behaviour. We propose a method based on abstraction and composition to model the services and their collaboration within a distributed system. In order to demonstrate suitability and usefullness of Petri net, we modelled a Banking as a Service (BaaS) application and analysed it to verify desired system properties.

The rest of the paper is organised as follows. In Section II we introduce Petri nets, give some background, give the formal definitions and illustrate some small examples. Section III explains our proposed method for modelling and analysing

2376-9521/23/$31.00 ©2023 IEEE
DOI 10.1109/SEAA60479.2023.00047

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

microservice-based systems with Petri nets. Section IV is devoted to our case study. Section V concludes the paper.

II. Petri nets

Petri nets were introduced in 1962 by Carl Adam Petri in [10] and have been widely studied ever since [11]–[14]. Petri nets are a well-established formalism for modelling concurrent systems. A well-defined mathematical theory together with a graphical representation of dynamic behaviour of systems, make them a widely used formalism for modelling and analysis of concurrent and distributed systems in various areas such as computer science, software engineering, business process management, control theory, bioinformatics, transportation systems, etc.

Microservice-based systems are characterised by their distributed nature and the need to coordinate the behaviour of multiple services. Petri nets are particularly suitable for modelling such systems because they allow for the representation of concurrency, synchronisation, and communication between services in a clear and intuitive way.

A. Preliminaries

In this section, we recall the basic definitions of Petri net theory.

A *net* is a triple $N = (P, T, F)$, where P and T are disjoint sets, and $F \subseteq (P \times T) \cup (T \times P)$ is called the *flow relation*.

We adopt the usual graphical notation: places are represented by circles, transitions by boxes and the flow relation by arcs. The set of elements of a net will be denoted by $X = P \cup T$.

The *preset* of an element $x \in X$ is ${}^\bullet x = \{y \in X | (y, x) \in F\}$; the *postset* of x is $x^\bullet = \{y \in X | (x, y) \in F\}$; the *neighborhood* of x is given by ${}^\bullet x^\bullet = {}^\bullet x \cup x^\bullet$. These notations are extended to subsets of elements in the usual way: i.e., let $A \subseteq X, {}^\bullet A = \bigcup_{x \in A} {}^\bullet x$.

For any net N we denote the *in-elements* of N by ${}^\circ N = \{x \in X : {}^\bullet x = \emptyset\}$ and the *out-elements* of N by $N^\circ = \{x \in X : x^\bullet = \emptyset\}$. When in-elements or out-elements are only places, we call them respectively, *in-places* or *out-places*.

A net $N' = (P', T', F')$ is a *subnet* of $N = (P, T, F)$ if $P' \subseteq P, T' \subseteq T$, and $F' = F \cap ((P' \times T') \cup (P' \times T'))$. Given a subset of elements $A \subseteq X$, we say that $N(A)$ is the *subnet of N identified* by A if $N(A) = (P \cap A, T \cap A, F \cap (A \times A))$.

An (ordinary) Petri net system $\Sigma = (P, T, F, m_0)$ is defined by a net (P, T, F), and an initial marking $m_0 : P \to \mathbb{N}$. The elements of P are called *places*, the elements of T are called *transitions*. A net is finite if the sets of places and of transitions are finite. A *marking* is a map $m : P \to \mathbb{N}$. Markings represent global states of a net. A marking m is safe if $m(p) \in \{0, 1\}$ for all $p \in P$. A transition t is *enabled* at a marking m, denoted $m[t\rangle$, if, for each $p \in {}^\bullet t$, $m(p) > 0$. Let t be enabled at m; then, t can fire in m producing the new marking m', denoted $m[t\rangle m'$ and defined as follows:

$$m'(p) = \begin{cases} m(p) - 1 & \text{for all } p \in {}^\bullet t \setminus t^\bullet \\ m(p) + 1 & \text{for all } p \in t^\bullet \setminus {}^\bullet t \\ m(p) & \text{in all other cases} \end{cases}$$

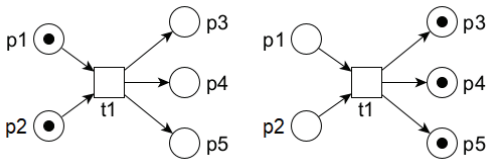


Fig. 1. Firing of a transition

In Fig. 1, an ordinary Petri net is illustrated. Transition t_1 has 2 pre-conditions, p_1, p_2, and 3 post-conditions p_3, p_4, p_5. It is enabled when both its pre-conditions are marked, i.e. have a token. When t_1 fires, it consumes the tokens from its pre-conditions and put tokens into its post-conditions.

A marking q is *reachable* from a marking m if there exist transitions $t_1 \dots t_{k+1}$ and intermediate markings $m_1 \dots m_k$ such that: $m[t_1\rangle m_1[t_2\rangle m_2 \dots m_k[t_{k+1}\rangle q$. The set of markings reachable from m will be denoted by $[m\rangle$. If all the markings in $[m_0\rangle$ are safe, then $\Sigma = (P, T, F, m_0)$ is said to be *1-safe* (or, shortly, safe). A marking of a 1-safe Petri net can be simply written as the list of marked places. For example, in Fig. 1, the initial marking, m_0 is (p_1, p_2). The marking after firing of t_1 is (p_3, p_4, p_5). Let $t_1, t_2 \in T$ and $m \in [m_0\rangle$, t_1 is *in conflict with* t_2 at m if they are both enabled at m and the firing of one of them disables the other one.

A transition is called *dead* at a marking m if it is not enabled at any marking reachable from m. A reachable marking m is called *dead* if no event is enabled at m. A Petri net system is *deadlock-free* if no reachable marking is dead. An transition t is *live* if, for each reachable marking m, there is a marking m' reachable from m that enables t. A Petri net system is *live* if all its transitions are live. A live net system is deadlock-free however vice versa is not correct. A deadlock-free net system can have events that are not live.

In the rest of the paper, we will consider 1-safe Petri nets, in which the underlying nets are finite and all transitions have non-empty presets, i.e., all have input places.

In Fig. 2, two Petri net systems are illustrated. Both nets are 1-safe. Net Σ_1 is deadlock-free and live. In Σ_1, t_1 is enabled at the initial marking. Transition t_1 consumes the token in $p1$ and puts a token in both $p2$ and $p3$. In other words, the marking reached after firing of t_1 is $(p2, p3)$. At marking $(p2, p3)$, t_2 and t_3 are concurrently enabled. The two transitions are causally independent so they can fire independently in any order. There are three possible scenarios for their firings. $t2$ can fire before $t3$, $t3$ can fire before $t2$ and they can fire simultaneously. Thus, the markings $(p2, p5)$, $(p3, p4)$ and $(p4, p5)$ are reachable by firing of $t2$ and $t3$. Transition t_4 models a synchronisation. It is only enabled after

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

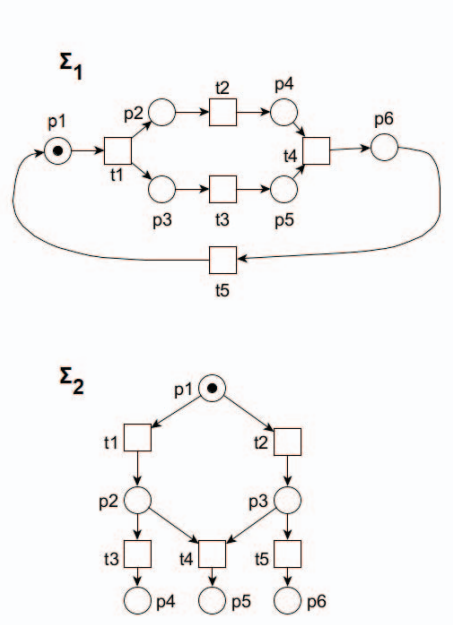


Fig. 2. Example Petri net systems

both concurrent transitions fire. In this net, we also see causal relation. For example, both t_2 and t_3 are causally dependent on transition t_1. Transition t_4 is causally dependent on t_2, t_3 and t_1. Transitions t_1, $\{t_2, t_3\}$, t_4 and t_5 are partially ordered, i.e. t_2 and t_3 are independent so can fire in any order after t_1 before t_4 but the rest of the transitions are in sequential order.

In Σ_2 of Fig. 2, t_1 and $t2$ are in conflict at the initial marking. They are both enabled at the same time but only one can fire since they share an input place and firing one will consume the token and disable the other transition. Thus, if $t1$ fires, the new marking reached will be $(p2)$, whereas if $t2$ fires the new marking reached will be $(p3)$. Notice that transition $t4$ needs two tokens to be enabled, one from $p2$ and one from $p3$. However, these two places are never marked together since they are marked by conflicting transitions. As a result, there is no reachable marking which enables $t4$, i.e. $t4$ is a dead transition. The net also has some dead markings such as $(p4)$ and $(p6)$. These markings do not enable any transitions. After firing of $t3$ or $t5$, the net reaches a deadlock.

III. Modelling and analysis of microservice-based systems with Petri nets

In this work, we propose using 1-safe Petri nets for modelling and verification of microservice based systems. We define a modelling method based on abstraction and composition which offers simple and efficient formal verification of a complex system.

Petri nets are known to be very powerful for modelling distributed and concurrent systems and they have been studied extensively so that there is a rich theory and a wide collection of tools. They can model conflicts, concurrency, synchronisation, causality and more complex relationships between entities of a system. These entities can be components of the whole system which are represented via subnets, actions performed in a system represented via transitions and local states which represent the conditions and effects of actions. By examination of Petri net model of a system, we can deduce properties like deadlock freedom, liveness, reversibility, boundedness, state machine decomposability, etc. By using the existing tools, we can simulate the nets to observe their behaviour, analyse the nets' structural and behavioural properties and we can perform model checking. Some of these tools are TINA [15], PIPE [16], CPN tool [17], Great SPN [18] and LoLA [19].

In formal verification of concurrent systems, liveness and safety properties are used for specification of behavioural properties which are then verified by performing various techniques. Liveness properties simply say that "good things happen" whereas safety properties say that "bad things do not happen". Correctness and reliability of a system can be formulated by combination of safety and liveness properties.

In order to verify such behavioural properties of the system, we can use reachability analysis such that a property is represented as a (global) state and then via performing a reachability analysis the property can be verified. Some states might be desired to always be reachable because they represent a desirable property (liveness properties), whereas some states must be not reachable in the system since they represent a faulty or unsafe situation (safety properties). An example of a liveness property could be account balance increase, i.e., for any transaction involving a deposit, the account balance should increase. An example of a safety property could be no overdrawn property, i.e., the system should ensure that no account balance goes below zero at any point during the transaction process.

For more complex properties that cannot be simply represented as a reachable state, we can use model checking methods. One method is to use linear-time temporal logic (LTL) for specification of a property to be verified. There are many tools available for performing model checking on different classes of Petri nets by using various techniques.

In this work, we propose a method for modelling microservice based systems based on abstraction and composition. A microservice based software system can include many microservices working together and communicating in different ways. However, the internal details of a service is not known by the other services. While modelling the system we consider the flow and the relationships between the components independently from the internal details of each service. Thus, we model each service separately in detail and we model the overall system by omitting the details of these services but focusing on their relationships, how they work together and how they communicate with each other. In the overall model, the work of a service is reduced to a single transition. To be able to do this reduction, it must be possible to simulate the service net by a single transition of the composed overall net. Below we define the necessary conditions to map

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

a service net to a transition of the composed overall net.

Let $\Sigma_s = (P_s, T_s, F_s, m_{0s})$ be the net to be reduced and Σ_o be the overall net. Σ_s can be reduced to a transition of $\Sigma_o = (P_o T_o, F_o, m_{0o})$ if and only if the following conditions hold.

- Σ_s is 1-safe,
- $(|^\circ\Sigma_s| = 1) \wedge (^\circ\Sigma_s \in P_s)$ and $(|\Sigma_s{}^\circ| = 1) \wedge (\Sigma_s{}^\circ \in P_s)$,
- At m_{0s} $m(^\circ\Sigma_s) = 1 \wedge \forall p \in P_s \setminus \{^\circ\Sigma_s\} \quad m(p) = 0$ and $(m(^\circ\Sigma_s) > 0 \implies \forall p \in P_s \setminus \{^\circ\Sigma_s\}, \quad m(p) = 0) \wedge (m(\Sigma_s{}^\circ) > 0 \implies \forall p \in P_s \setminus \{\Sigma_s{}^\circ\}, \quad m(p) = 0)$.

In other words, the net to be reduced must be 1-safe. It must have only one in-element and only one out-element which are places. At the initial marking only the in-place is marked. When the in-place is marked, all other places must be empty and when the out-place is marked all the other places must be empty. These conditions ensure that the whole service net can act as one single transition in an abstract view.

Let $t \in T_o$, if the above conditions hold for Σ_s, then the whole net can be reduced and mapped to the overall net in the following way.

- The in-element $^\circ\Sigma_s$ is mapped to $^\bullet t$,
- the out-element $\Sigma_s{}^\circ$ is mapped to $t^\bullet$,
- $(P_s \cup T_s) \setminus (^\circ\Sigma_s \cup \Sigma_s{}^\circ)$ is mapped to the transition t.

The above method enables a simpler analysis by reducing the size and complexity of the overall net modelling the overall flow and communication among the components of the system.

IV. Case Study

A. Banking as a service

To demonstrate how to use our proposed method and to investigate the usability and the benefits of Petri nets for modelling and verification of microservice based systems, we performed a case study. In this study, we modelled a part of Banking as a Service (BaaS) application.

Banking as a Service (BaaS) is a type of financial technology that enables non-bank financial institutions, businesses, and individuals to access various banking services through the use of application programming interfaces. Traditionally, banking services were provided only by banks, but with BaaS, third-party providers can offer banking services such as payment processing, account creation and management, fraud detection, and lending services. These services can be customised to meet the specific needs of the company or individual using the BaaS provider's services.

As any other finance related software system, it is critical to ensure the correctness and reliability of BaaS applications. In particular, using microservice based approach brings a big number of components working together and updating the system simultaneously. This makes the testing, monitoring and maintaining harder. Applying formal methods, especially via Petri nets, has many advantages in tackling these problems. These advantages include having a visual representation of the system, being able to simulate the net to observe the behaviour, tackling problems before developing the application, providing guarantees for the desired system properties, computing all possible executions and all possible reachable states and making generation of test cases easier.

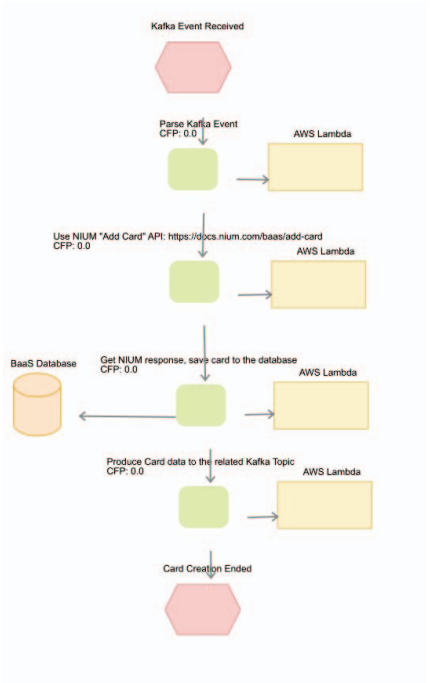


Fig. 3. eEPC diagram of card creation

The eEPC diagrams in Fig. 3 and Fig. 4 were created by the development team. Based on our discussions and the existing eEPC diagram, we improved the models by including missing details, we identified the services and modelled them separately and by applying our composition method, we modelled the overall scenario with 1-safe Petri nets.

B. The Petri net model

As a case study to show how 1-safe Petri nets can be used for formal modelling and analysis of microservice-based systems, we modelled and analysed a part of a banking as a service application as explained above. We used the tools PIPE [16] and TINA [15] for this analysis.

Here we present Petri net models for the system described above. We have three Petri net models such that the net in Fig. 5 models the service taking care of customer creation; the net in Fig. 6 models the service taking care of card creation; and the net in Fig. 7 models the overall scenario. It is worth to note that the Petri nets models and eEPC diagrams do not comply with each other completely because based on discussions with the team, some missing details were added and models were improved. Some of the transitions and places are given explanatory names whereas some are only labelled as p_i, t_j. Transitions stand for actions performed whereas places represent local states which are changed by actions.

First of all, we must examine the two nets modelling the services. There are three requirements that must be satisfied by

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

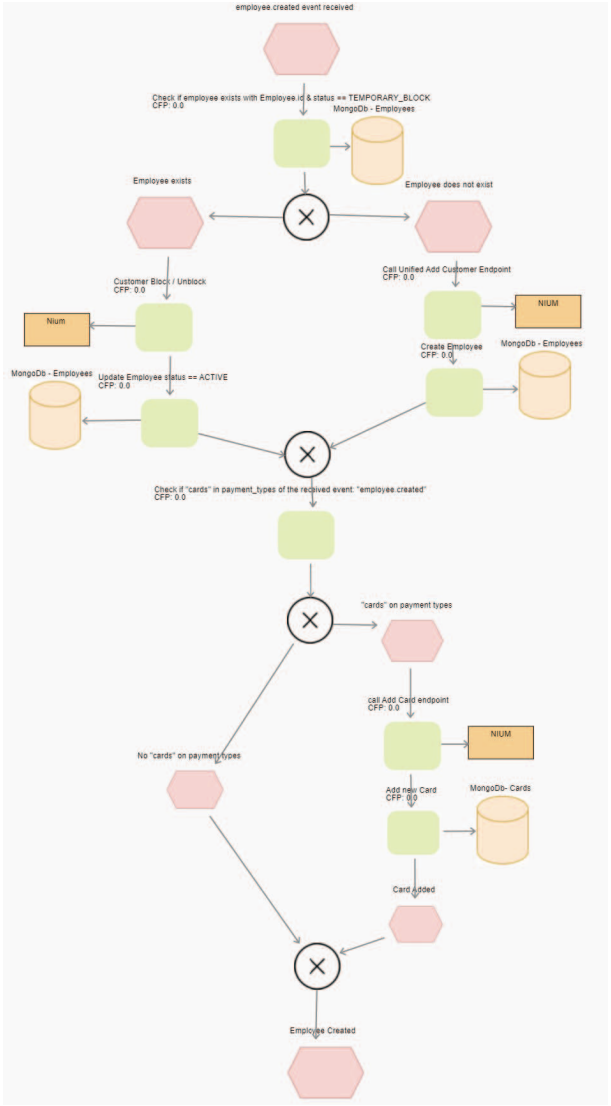


Fig. 4. eEPC diagram of BaaS customer (here called employee) creation

the nets to be able to apply the abstraction method proposed above. Both nets are 1-safe meaning that number tokens in a place can be at most 1. We see that both nets start with an initial place (p_1 for both nets) and end with a final place (p_{10} in Fig 5, p_8 in Fig. 6). When we further analyse the nets, we see that in both nets the only deadlock is the final state which represent the completion of the work done by the service. As required, there is no other deadlock within the net. Moreover, when the final state is reached, there are no other tokens remained in the net. As a result the nets can be represented as a single transition in the overall model. Note that p_1 and p_{10} in Fig. 5 are mapped to p_7 and p_{10} in Fig. 7 while the part between the initial and final places of the net in Fig. 5 is mapped to transition t_6 in Fig. 7. Similarly, p_1 and

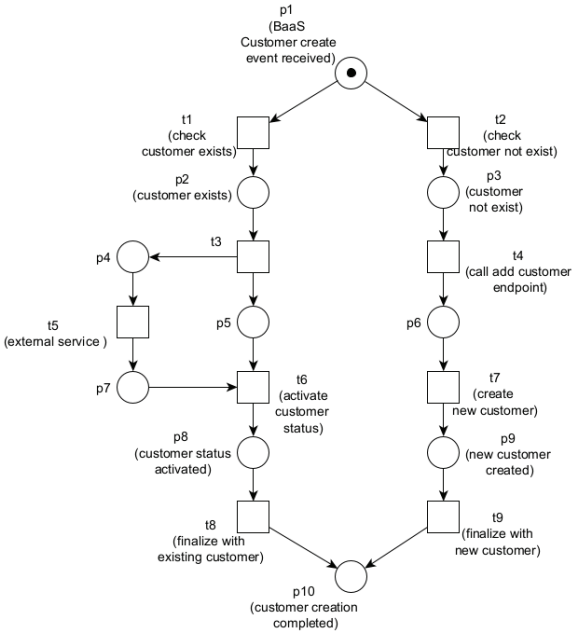


Fig. 5. Model of creating a new customer

p_8 in Fig. 6 are mapped to p_{15} and p_{16} in Fig. 7 while the part between the initial and final places of the net in Fig. 6 is mapped to transition t_{12} in Fig. 7. A detailed analysis of each service can be done separately while the communication and relation between the services and the the overall system behaviour can be formally analysed on the composed model (Fig. 7). In the following we will first analyse the service nets in more detail and then we will analyse the overall net.

In Fig. 5, customer creation service starts working when p_1 is marked. This means that a BaaS customer create event is received. The service first checks if the customer already exists in the system. Here the process splits into two conflicting paths. If the customer exists, the left hand side path is followed, otherwise the right hand side path is followed. If the customer already exists then the customer status is activated. Note that firing of transition t_3 creates two parallel paths. In one path, an external service is called while in the parallel path the internal workflow of the modelled service continues. However, the details of this part is omitted and represented as only one place which remains true until the external service finishes its job. Transition t_6 is a synchronisation between the external service and the modelled service. The process ends when place p_{10} is marked. Note that this place can be marked by either t_8 or t_9 but not both of them at the same time. Clearly this net is not live since it reached a deadlock in the final state. However, this is a desired property of the system, so it does not have a negative meaning. In the following, we will see that good states are guaranteed to be reachable whereas bad states are guaranteed to be not reachable in the net.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

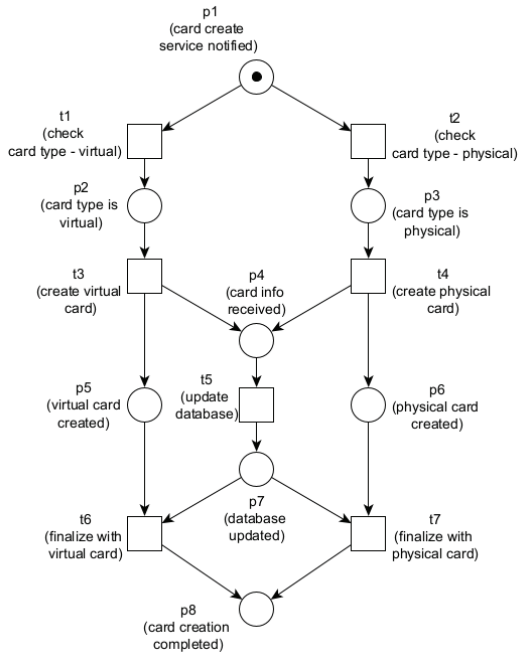


Fig. 6. Model of card creation service

Although the model is already simple, we will analyse it for some desired properties. For this, we first do reachability analysis for the properties that can be specified by a reachable marking. A liveness property that the net must satisfy is that customer creation will be completed. This can be represented as reachability of a marking in which p_{10} is marked. In fact, the reachability analysis shows that this property is satisfied by a reachable marking which is the final state (p_{10}). An example of a safety property that must be satisfied by the system is as follows: "customer exists" and "new customer created" cannot be true at the same time. According to this, any state in which p_2 and p_9 are marked together is a bad state and must not be reachable. In fact, reachability analysis shows that the two places are never marked together in a reachable state.

Above reachability analysis proves some safety and liveness properties to show the correctness of the system. However, some properties are more complicated and cannot be simply represented as a reachable marking. For the specification of such properties, we can use LTL and perform LTL model checking on the model. Below, we write some properties, their corresponding LTL formulae and the analysis results.

- if customer exist (p2) is not true, new customer will be created.
  - LTL formula: $\Box\neg p_2 \implies \Diamond p_9$
  - result: true
- For all executions of the system it is true that if a BaaS customer create event is received then either a new customer is created or an existing customer is activated but not together.
  - LTL formula: $\Diamond p_1 \implies (\Diamond p_8 \vee \Diamond p_9) \wedge \Box\neg(p_8 \wedge p_9)$
  - result: true

The net in Fig. 6 models the service which handles the card creation. After the card creation service is notified, the card type is checked. The card type can be either virtual or physical but not both. The flow continues in two conflicting paths. In both cases the database is updated in a parallel path. In the end the card creation is completed with a virtual card or a physical card. The net is not live by design, since it ends in a final state in which card creation is completed.

The reachability of the final state is desired. In fact, reachability analysis shows that it is reachable. One of the good property that is desired to be satisfied is that card info will be received in the system. To this aim, we check if a state in which p_4 is marked is reachable. Reachability analysis shows that the reachable states (p_4, p_5) and (p_4, p_6) satisfy this. There are some cases that we want the system to avoid for the safety of the system. For example, if the card type is virtual place p_6 should not be marked. The reachability analysis proves that there is no state in which p_2 and p_6 are marked.

Although the above analysis proves some good properties, we need stronger ones to ensure the correctness of the system. The fact that p_2 and p_6 are never marked in the same reachable state is a good property but it is also important that if p_2 is marked at a reachable marking, p_6 should never be marked in any reachable marking. Below we write some properties, their corresponding LTL formulae and the analysis results.

- If card type is virtual, a physical card should not be created (p_6 should not be marked).
  - LTL formula: $\Diamond p_2 \implies \Box\neg p_6$
  - result: true
- If card creation service is notified then database will be updated and card creation will be completed.
  - LTL formula: $\Diamond p_1 \implies (\Diamond p_7 \wedge \Diamond p_8)$
  - result: true

The net in Fig. 7 models the overall scenario in a way that the details of the jobs done by the services are not included in the model. The initial marking $(p1, p3, p6, p13)$ represents the state where a customer creation request arrived and three services are available to be used during the execution of the whole customer and card creation process. These services are "Customer Create Service", "Know Your Customer Service" and "Card Create Service". Each of these services are modelled by a single transition in the overall model. More specifically, t_6 is the abstraction of customer creation service, t_7 is the abstraction of know your customer service and t_{12} is the abstraction of card create service. Refinement of t_6 is the net in Fig. 5 whereas refinement of t_{12} is the net in Fig. 6. Note that the refined models satisfy the abstraction requirements explained above. By analysing the overall model (Fig. 7), we can see that t_5 is independent from t_6 and t_{12}. This means that know your customer service can run concurrently with customer creation service and card creation service. However, we can also see from the net that t_{12} is causally dependent

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

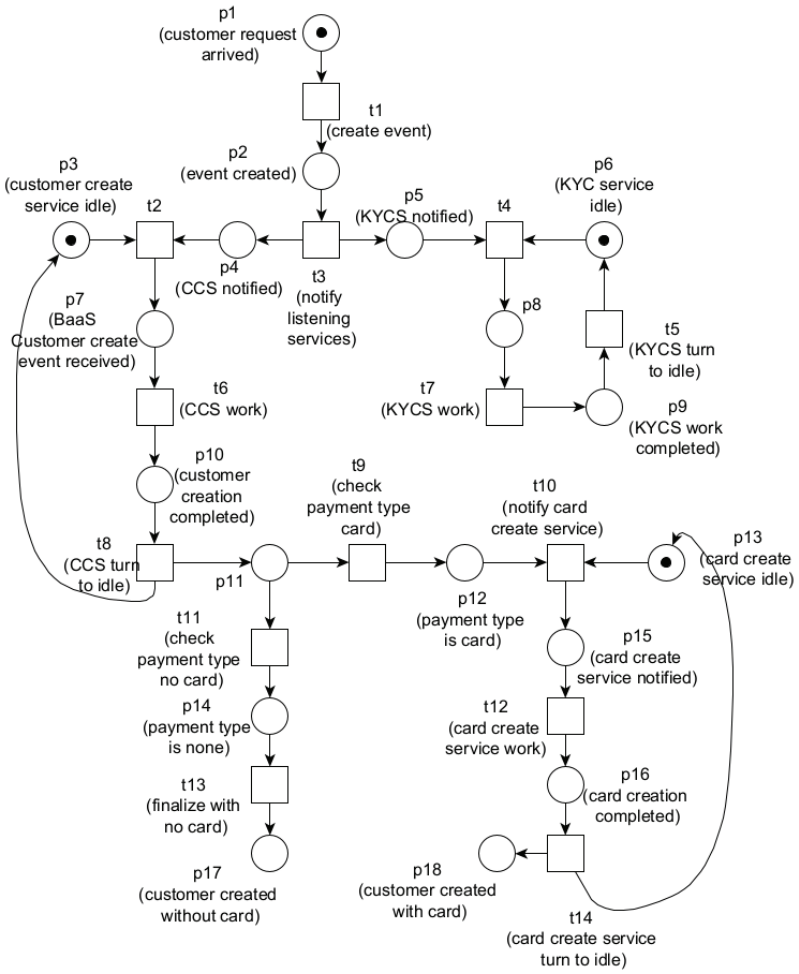


Fig. 7. Overall view of creating a new BaaS customer

on t_6. This means that card creation service needs to wait until the customer creation service finishes its job. It is also deduced from the net, depending on the customer information, card creation service might not need to work for each customer since the customer can choose not to have a card. In this case, the process ends with creating a customer without a card, i.e. in the final marking p_{17} is marked instead of p_{18}.

When we further investigate the net, we confirm that the net is 1-safe, i.e. number of tokens in a place cannot be more than 1. This allows us to interpret each place as a local condition which can either be true or false (true when it is marked, false when not marked). Furthermore, from the analysis we see that the net is a free-choice net, which means that there is no confusion. The net is not live since it reaches a deadlock. However this is expected and does not have a negative meaning since the net has two possible final states to end the process. The only deadlocks in the net are these final states, i.e. $(p_3, p_6, p_{13}, p_{17})$ and $(p_3, p_6, p_{13}, p_{17})$ representing the new customer creation with a card and new customer creation without a card. In fact, although these states are deadlocks, their reachability is necessary.

We performed reachability analysis on the net. One of the properties we require to be satisfied by the system is that it reaches a state in which customer creation is completed. This property can be expressed by a reachable marking. By checking the system for reachability of a global state in which p_{10} is marked, we can confirm that the property is satisfied. By the reachability analysis we see that multiple reachable states satisfy this property, e.g. $(p_5, p_6, p_{10}, p_{13})$ and (p_9, p_{10}, p_{13}). A safety property to support the correctness of this system is a confusion in the customer creation finalisation. It can't be true that a customer creation is completed with card and without a card at the same time. This property can be written as a global state of the system in which p_{17} and p_{18} are marked together.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.

When we analyse the net for reachability of such states we see that such a state is not reachable in this system. So we can say that the safety property is satisfied.

However, the safety property we confirmed above is not enough to verify that customer creation because although it proves that p_{17} and p_{18} are never marked together in the same global state, it does not guarantee that in every execution of the system only one type of finalisation is done (either with a card, or without a card). This is a property that we can represent as an LTL formula. Below, we write some properties as LTL formulas and the result of model checking for these formulas.

- For all executions of the system it is true that p_{17} (customer created without card) and p_{18} (customer created with card) are not true in the same execution.
  - LTL formula: $\neg(\Diamond p_{17} \wedge \Diamond p_{18})$
  - result: true
- If the card type is "none", card creation should not be performed.
  - LTL formula: $\Diamond p_{14} \implies \Box\neg p_{16}$
  - result: true
- If a customer request arrives, eventually customer should be created without or with card.
  - LTL formula: $\Diamond p_1 \implies \Diamond(p_{17} \vee p_{18})$
  - result: true

V. Conclusions and future work

In this work, we propose using Petri nets for modelling microservice-based systems. We define a method based on abstraction and composition which enables modelling and analysing a complex microservice-based system. In this method, we can model each microservice separately and analyse them to formally verify the models for the desired properties. We can model the overall complex system by reducing the service nets into a single transition and simply focusing on the relationships between the services and the overall system behaviour. By this method, we can simulate the models to observe their behaviours, we can analyse structural and behavioural properties of nets and perform reachability analysis and model checking on the nets to formally verify correctness and reliability of the microservice-based system.

To demonstrate our method and show the suitability and usefullness of Petri nets, we performed a case study in which we modelled and analysed a BaaS application. This case study showed that in fact 1-safe Petri nets provide a basis for analysis and verification of microservice-based systems. Various liveness and safety properties can be specified and verified with 1-safe Petri nets. All reachable states can be computed and critical states can be identified. In addition to formal verification, Petri net models can help with generating test cases for testing the application. Its graphical representation enables people who are not necessarily familiar with formal methods to understand and use the models.

As future work, we plan to expand the case study by including more components and do further analysis. One of the possible extension is to explore security analysis by considering information flow among components of a microservice-based system that is modelled with Petri nets. We also would like to explore different Petri net classes especially higher level Petri nets, such as coloured Petri nets in order to integrate data values into the models. Moreover we plan to explore nets within nets paradigm which allows tokens to be nets.

References

[1] J. Thönes, "Microservices," *IEEE Software*, vol. 32, no. 1, pp. 116–116, 2015.

[2] J. Bonér, *Reactive Microservices Architecture*. O'Reilly Media, Inc., 2016.

[3] N. Alshuqayran, N. Ali, and R. Evans, "A systematic mapping study in microservice architecture," in *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications*. IEEE, Nov. 2016, pp. 44–51.

[4] P. Di Francesco, P. Lago, and I. Malavolta, "Architecting with microservices: A systematic mapping study," *Journal of Systems and Software*, vol. 150, pp. 77–97, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0164121219300019

[5] H. Ünlü, B. Bilgin, and O. Demirörs, "A survey on organizational choices for microservice-based software architectures," *Turkish J. Electr. Eng. Comput. Sci.*, vol. 30, no. 4, pp. 1187–1203, 2022. [Online]. Available: https://doi.org/10.55730/1300-0632.3843

[6] H. Ünlü, S. Tenekeci, A. Yildiz, and O. Demirörs, "Event oriented vs object oriented analysis for microservice architecture: An exploratory case study," in *47th Euromicro Conference on Software Engineering and Advanced Applications, SEAA 2021, Palermo, Italy, September 1-3, 2021*, M. T. Baldassarre, G. Scanniello, and A. Skavhaug, Eds. IEEE, 2021, pp. 244–251. [Online]. Available: https://doi.org/10.1109/SEAA53835.2021.00038

[7] J. Ortiz, V. Torres, and P. Valderas, "Microservice compositions based on the choreography of BPMN fragments: facing evolution issues," *Computing*, vol. 105, no. 2, pp. 375–416, 2023. [Online]. Available: https://doi.org/10.1007/s00607-022-01128-8

[8] B. F. van Dongen, W. M. P. van der Aalst, and H. M. W. Verbeek, "Verification of epcs: Using reduction rules and Petri nets," in *Advanced Information Systems Engineering*, O. Pastor and J. Falcão e Cunha, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 372–386.

[9] A. Lyazidi and S. Mouline, "Formal verification of UML state machine diagrams using Petri nets," in *Networked Systems - 7th International Conference, NETYS 2019, Marrakech, Morocco, June 19-21, 2019, Revised Selected Papers*, ser. Lecture Notes in Computer Science, M. F. Atig and A. A. Schwarzmann, Eds., vol. 11704. Springer, 2019, pp. 67–74. [Online]. Available: https://doi.org/10.1007/978-3-030-31277-0_5

[10] C. A. Petri, "Kommunication mit automaten," Ph.D. dissertation, Technischen Hoschule Darmstadt, 1962.

[11] T. Murata, "Petri nets: Properties, analysis and applications," *Proc. IEEE*, vol. 77, no. 4, pp. 541–580, 1989.

[12] J. L. Peterson, "Petri nets," *ACM Comput. Surv.*, vol. 9, no. 3, pp. 223–252, 1977.

[13] W. Reisig, *Understanding Petri Nets - Modeling Techniques, Analysis Methods, Case Studies*. Springer, 2013.

[14] J. Desel and G. Juhás, ""what is a Petri net?"," in *Unifying Petri Nets, Advances in Petri Nets*, ser. Lecture Notes in Computer Science, H. Ehrig, G. Juhás, J. Padberg, and G. Rozenberg, Eds., vol. 2128. Springer, 2001, pp. 1–25. [Online]. Available: https://doi.org/10.1007/3-540-45541-8_1

[15] TIme petri Net Analyzer. [Online]. Available: https://projects.laas.fr/tina/index.php

[16] Platform Independent Petri net Editor 2. [Online]. Available: https://pipe2.sourceforge.net/

[17] A tool for editing, simulating, and analyzing colored Petri nets. [Online]. Available: https://cpntools.org/

[18] GRaphical Editor and Analyzer for Timed and Stochastic Petri Nets. [Online]. Available: https://cpntools.org/

[19] (LoLA) - A Low Level Petri Net Analyzer. [Online]. Available: https://theo.informatik.uni-rostock.de/theo-forschung/tools/lola/

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 08,2026 at 14:43:13 UTC from IEEE Xplore. Restrictions apply.